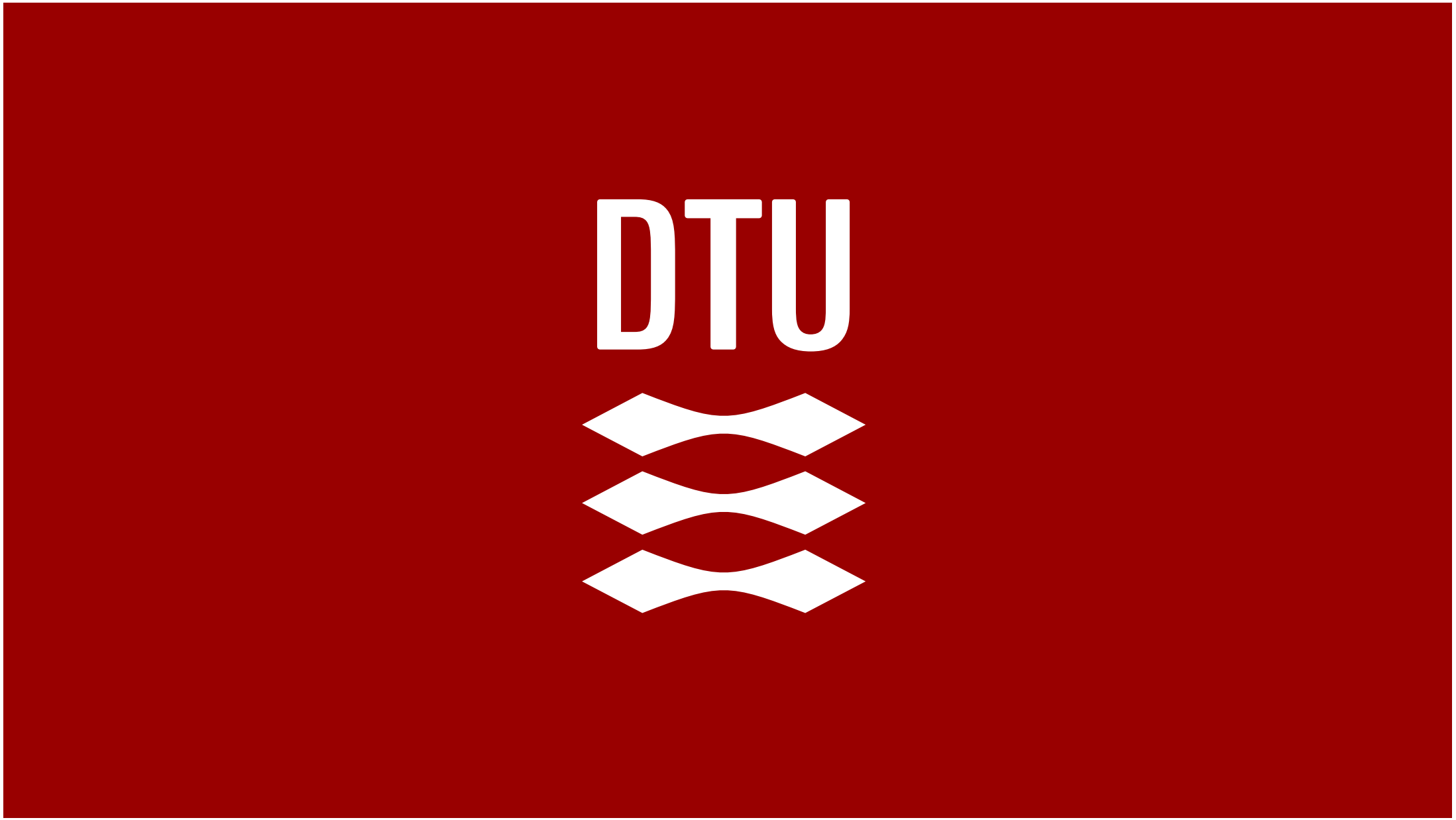

DTU

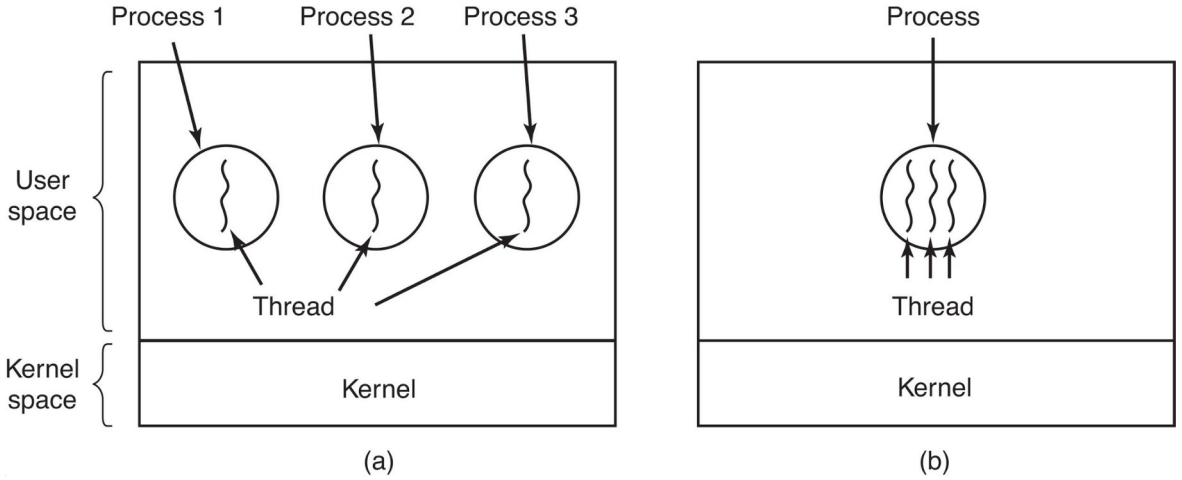
02159 – Operating Systems

Week 7: Inter-Process Communication (IPC)

Xenofon (Fontas) Fafoutis
Professor
xefa@dtu.dk
www.compute.dtu.dk/~xefa

Interprocess Communication

- How do different threads **communicate**?
  - Threads of different processes (**inter-process**)
  - Threads within the same process (**intra-process**)



Ways to Communicate

- **Shared Memory**
  - Global variables, heap memory (threads only)
  - Shared memory in the kernel
  - Files

- **Messages and Data Streams**
  - Signals
  - Pipes
  - Sockets (UDP, TCP, UNIX)

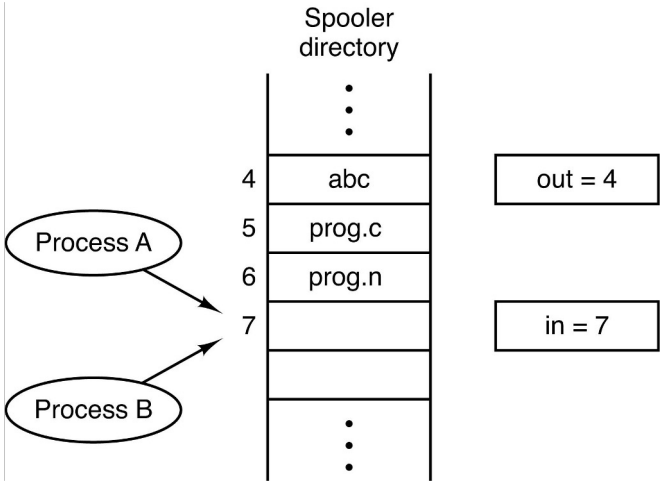
Communication through Files

- The **bank account** example
  - Your bank account balance is saved on a **file**
  - A periodic process **withdraws** an amount
  - Another periodic process **deposits** an amount

- What is happening here?

Murphy's Law

- A and B want to print a file
- A adds the file in the printing queue and increments the ***in*** counter
- B adds the file in the printing queue and increments the ***in*** counter
- Imagine the following happens:
  - A reads ***in*** and stores the value in a register
  - Scheduler stops A and starts B
  - B reads ***in*** and stores the value in a register
  - B enters file in position 7 and increments the counter
  - Scheduler stops B and starts A
  - A thinks the next available slot is 7
  - A enters file in position 7 and increments the counter
  - A overwrites the file of B
  - A prints the file
  - B is still waiting



Race Condition

- **Race Condition** is the situation where the **outcome** depends on the **timing** of process/thread execution (scheduling)

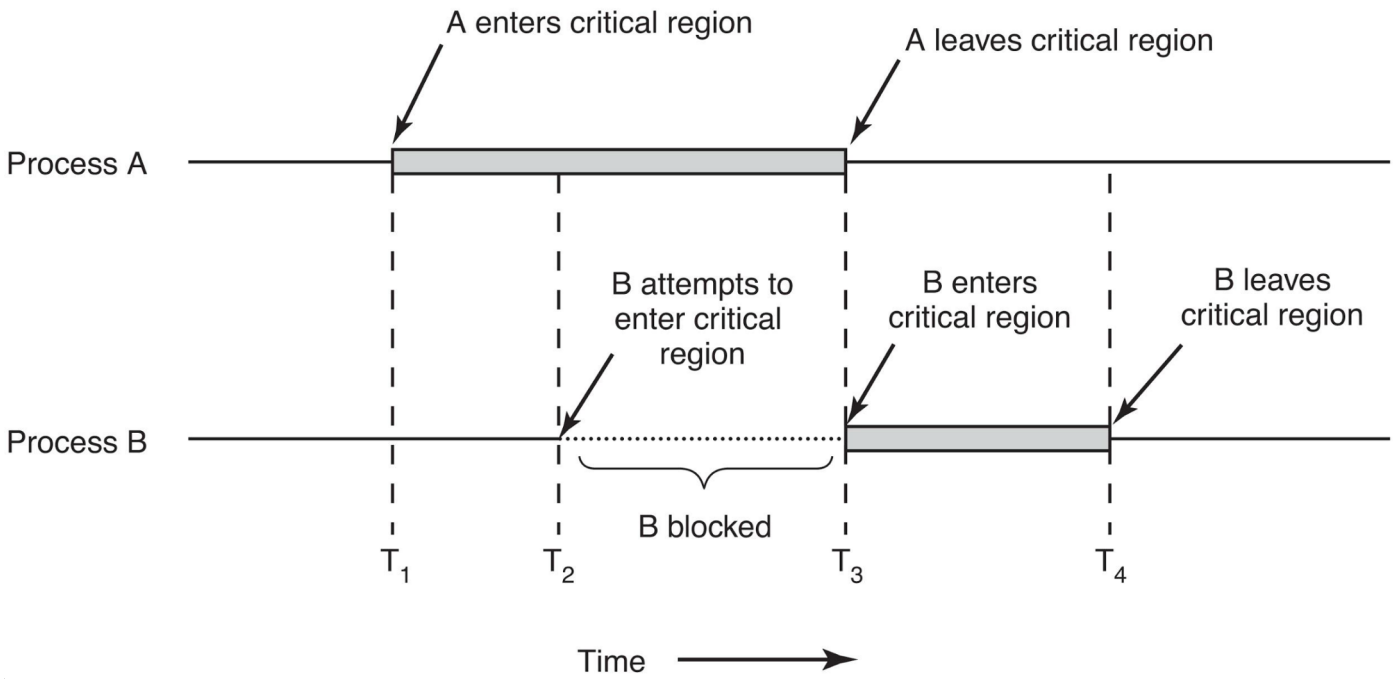
- Scheduling is effectively **random** (unpredictable and uncontrollable) from the perspective of process/thread
- A race condition leads to unexpected **non-deterministic** behaviour
  - Bugs that appear **rarely** and are **hard to explain**
  - Plain testing is **not** sufficient

How do we avoid Race Conditions?

- **Critical Regions**
  - The parts of the programme where the shared memory, shared file, or other shared resource is **accessed**
- We avoid race conditions with **mutual exclusion**
  - Ensuring that when one process/thread is in the critical region, no other process/thread can enter the critical region
  - **Locking** (or acquiring) and **unlocking** (or releasing) the resource

- Example: A public toilet
  - Goal: Up to one person can use the toilet at any given time
  - Solution: Mutual exclusion with a lock
    - A person can enter the toilet **only** when it is **unlocked**
    - When inside the toilet, the person **locks** it before using it
    - Others cannot enter a locked toilet, they must **wait**
    - The person **unlocks** the toilet when exiting making it available to the others

Mutual Exclusion using Critical Regions



Lock Variables

- Imagine process A and process B execute the following code
  - **lock** is a **shared variable**

- Is this **safe** from race conditions?

```
while(1) {
  if (lock == 0){
    lock = 1;
    critical_region();
    lock = 0;
    break;
  }
}
```

Lock Variables

- Imagine process A and process B execute the following code
  - **lock** is a **shared variable**
- Is this **safe** from race conditions?

```
while(1) {
  if (lock == 0){
    lock = 1;
    critical_region();
    lock = 0;
    break;
  }
}
```

No! A process might be stopped between the **if** statement and the **lock = 1;** statement.

Strict Alternation

- Process A and Process B take **turns**
  - When A exits the critical region gives the turn to B and vice versa
- Is this **safe** from race conditions?

```
while (TRUE) {
    while (turn != 0)        /* loop */ ;
    critical_region( );
    turn = 1;
    noncritical_region( );
}
```

```
while (TRUE) {
    while (turn != 1)        /* loop */ ;
    critical_region( );
    turn = 0;
    noncritical_region( );
}
```

Strict Alternation

- Process A and Process B take **turns**
  - When A exits the critical region gives the turn to B and vice versa
- Is this **safe** from race conditions?

```
while (TRUE) {
    while (turn != 0)        /* loop */ ;
    critical_region( );
    turn = 1;
    noncritical_region( );
}
```

```
while (TRUE) {
    while (turn != 1)        /* loop */ ;
    critical_region( );
    turn = 0;
    noncritical_region( );
}
```

Yes, but it has a problem. A process cannot enter the critical region **twice** in a row, thus a slow process would slow down a fast process. Also, a process may be stopped from entering the critical region with no reason!

DTU

Hardware-based Solutions

- The source of the problem is that the process can be preempted **between** the instruction that **checks** the lock variable and the instruction that **sets** the lock variable
- If checking and setting were **indivisible** or **atomic**, all would be fine!
- Can the Hardware help?

Disabling Interrupts

- **Disabling Interrupts**
  - No clock interrupts, no preemption, **no race conditions**
  - Allowing the user to disable interrupts can be catastrophic
    - We can however implement **locks** at the **kernel**
    - See Contiki-NG OS
  - Works only in **single-processor** systems

```
while(1) {
  disable_interrupts();
  critical_region();
  enable_interrupts();
}
```

Atomic Instructions

- **Atomic Instructions**
  - Lock variable do not work because the process/thread might be stopped in between testing and setting the lock
  - The TSL (Test and Set Lock) instruction ensures that testing and setting the lock is **indivisible** or **atomic**
  - The CPU executing the TSL instruction **locks the memory bus** to prevent other CPUs from accessing the memory

```
enter_region:
        TSL REGISTER,LOCK       | copy lock to register and set lock to 1
        CMP REGISTER,#0         | was lock zero?
        JNE enter_region        | if it was not zero, lock was set, so loop
        RET                     | return to caller; critical region entered

leave_region:
        MOVE LOCK,#0            | store a 0 in lock
        RET                     | return to caller
```

DTU

Spinlock and Busy Waiting

- **Busy waiting** is when a process/thread **continuously tests a lock** until it is unlocked
  - Continuously try to open the locked door of a public toilet
  - Problems?

Spinlock and Busy Waiting

- **Busy waiting** is when a process/thread **continuously tests a lock** until it is unlocked
  - Continuously try to open the locked door of a public toilet
  - Problems?
    - Not very polite (in case of a public toilet)
    - Wasting CPU resources (in case of processes/threads)

- A lock that uses busy waiting is called a **spinlock**
  - Spinlocks are useful in multi-processor systems in cases of very short waiting times

Mutex

- Mutexes are safe **locks** provided by the Operating System
  - They are based on **atomic instructions**
  - **No busy waiting**

```
mutex_lock:
        TSL REGISTER,MUTEX      | copy mutex to register and set mutex to 1
        CMP REGISTER,#0         | was mutex zero?
        JZE ok                  | if it was zero, mutex was unlocked, so return
        CALL thread_yield       | mutex is busy; schedule another thread
        JMP mutex_lock          | try again
ok:     RET                     | return to caller; critical region entered

mutex_unlock:
        MOVE MUTEX,#0           | store a 0 in mutex
        RET                     | return to caller
```

Avoiding Race Conditions with Mutexes

```
while(1) {
  mutex_lock();
  critical_region();
  mutex_unlock();
}
```

Mutexes in Pthreads

- Mutexes can protect shared variables (**global, heap**) among **threads within a process**
- Mutexes can also protect shared variables (**shared memory**) and files among threads in **different processes** (the mutex must be allocated in the shared memory)

Thread call	Description
Pthread_mutex_init	Create a mutex
Pthread_mutex_destroy	Destroy an existing mutex
Pthread_mutex_lock	Acquire a lock or block
Pthread_mutex_trylock	Acquire a lock or fail
Pthread_mutex_unlock	Release a lock

File Locks

- Mutual exclusion can also be ensured with **lock files**
  - The OS can ensure that **only one process** can open a file

```
while(1) {
  lockf(LOCK);
  critical_region();
  lockf(UNLOCK);
}
```

Back to the Bank Account Example

- We want to implement a function that sends money (e.g. MobilePay)

```
send_money(int a, int b, int amount) {
  mutex_lock(&mutex[a]);
  mutex_lock(&mutex[b]);
  account[a] = account[a] - amount;
  account[b] = account[b] + amount;
  mutex_unlock(&mutex[b]);
  mutex_unlock(&mutex[a]);
}
```

- Is this safe?

Back to the Bank Account Example

- We want to implement a function that sends money (e.g. MobilePay)

```
send_money(int a, int b, int amount) {
  mutex_lock(&mutex[a]);
  mutex_lock(&mutex[b]);
  account[a] = account[a] - amount;
  account[b] = account[b] + amount;
  mutex_unlock(&mutex[b]);
  mutex_unlock(&mutex[a]);
}
```

- Is this safe?
  - There is a risk for **deadlock** (A waits for B, while B waits for A)
    - `send_money(0,1,100)` in parallel with `send_money(1,0,200)`
  - To avoid deadlocks, we must lock resources always in the **same order**

The Producer-Consumer Problem

- Two processes share a **fixed-size buffer**
  - The producer puts items in the buffer
  - The consumer takes items out of the buffer
- If the buffer is **full**, the producer must **wait** for an item to be removed
- If the buffer is **empty**, the consumer must **wait** for an item to appear

- We want to avoid inefficient busy waiting

- The processes **synchronise** by signaling each other
  - If the buffer is **full**, the producer goes to **sleep**
  - The consumer wakes up the producer when an empty slot appears
  - If the buffer is **empty**, the consumer goes to **sleep**
  - The producer wakes up the consumer when an item appears

The Producer-Consumer Problem

```
#define N 100                                   /* number of slots in the buffer */
int count = 0;                                  /* number of items in the buffer */

void producer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          item = produce_item( );               /* generate next item */
          if (count == N) sleep( );             /* if buffer is full, go to sleep */
          insert_item(item);                    /* put item in buffer */
          count = count + 1;                    /* increment count of items in buffer */
          if (count == 1) wakeup(consumer);     /* was buffer empty? */
     }
}


void consumer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          if (count == 0) sleep( );             /* if buffer is empty, got to sleep */
          item = remove_item( );                /* take item out of buffer */
          count = count – 1;                    /* decrement count of items in buffer */
          if (count == N – 1) wakeup(producer); /* was buffer full? */
          consume_item(item);                   /* print item */
     }
}
```

Two Race Conditions

- The previous solution to the Producer-Consumer problem has **two** race conditions
- We need **mutual exclusion** in the shared resources (buffer, count)
- There is a race condition in the **sleep-wakeup signaling**
  - Buffer is **empty**, counter is 0
  - Consumer checks that the counter is 0, so the **if** statement is **true**
  - Scheduler stops consumer and starts producer
  - Producer adds an item, updates counter to 1, and sends **wakeup**
  - Consumer is awake, receives the wakeup signal and **ignores** it
  - Consumer continues execution and goes to **sleep**
  - Eventually Producer fills up the buffer and goes to **sleep**
  - Both **sleep forever**

Semaphores

- Invented by Dijkstra in 1965
- A **semaphore** is an integer (greater or equal to 0)

- Two **atomic** operations: **down** and **up**
  - If > 0, **down** decrements the semaphore and continues
  - If == 0, **down** blocks
  - **up** increments the value of the semaphore

- Managing a pool of N resources
  - **down** reserves a resource or blocks if no resource available
  - **up** releases a resource, making it available
  - A semaphore tracks only **how many** resources are free; it does **not** keep track of **which** of the resources are free.
  - A **binary semaphore** is when N=1

Solving The Producer-Consumer Problem

```
#define N 100                                   /* number of slots in the buffer */
typedef int semaphore;                          /* semaphores are a special kind of int */
semaphore mutex = 1;                            /* controls access to critical region */
semaphore empty = N;                            /* counts empty buffer slots */
semaphore full = 0;                             /* counts full buffer slots */

void producer(void)
{
    int item;

    while (TRUE) {                              /* TRUE is the constant 1 */
        item = produce_item( );                 /* generate something to put in buffer */
        down(&empty);                           /* decrement empty count */
        down(&mutex);                           /* enter critical region */
        insert_item(item);                      /* put new item in buffer */
        up(&mutex);                             /* leave critical region */
        up(&full);                              /* increment count of full slots */
    }
}


void consumer(void)
{
    int item;

    while (TRUE) {                              /* infinite loop */
        down(&full);                            /* decrement full count */
        down(&mutex);                           /* enter critical region */
        item = remove_item( );                  /* take item from buffer */
        up(&mutex);                             /* leave critical region */
        up(&empty);                             /* increment count of empty slots */
        consume_item(item);                     /* do something with the item */
    }
}
```

Semaphores and Mutexes

- Semaphores and Mutexes are similar but **different**
  - **Do not confuse them!**

- **Mutex**
  - It solves the problem of **mutual exclusion**
    - Use a mutex when you execute code that no other process/thread should execute at the same time
  - A mutex has an **owner**, only the owner can **up** the mutex
    - Lock (down) and unlock (up) are coupled
- **Semaphore**
  - It solves the problem of **signaling**
    - Use a semaphore when you want to sleep until some other process/thread wakes you up
  - A semaphore does not have an owner, typically another process/thread is going to **up** the semaphore
    - Sleep (down) and wake-up (up) are decoupled

Message Passing

- Processes can also communicate by **exchanging messages**
  - Applies to processes within the **same** computer
  - It also applies to processes the run in **different** computers
    - **Computer Networking**

- Based on two communication primitives
  - send(destination, &message);
    - Sends messages to an **address**
  - receive(source, &message);
    - **Blocks** until it receive message

- No race conditions, but networking introduces **new problems**
  - What if a messages do not arrive?
  - What if messages arrive out of order?

Solving The Producer-Consumer Problem

```
#define N 100                                   /* number of slots in the buffer */

void producer(void)
{
     int item;
     message m;                                 /* message buffer */

     while (TRUE) {
          item = produce_item( );               /* generate something to put in buffer */
          receive(consumer, &m);                /* wait for an empty to arrive */
          build_message(&m, item);              /* construct a message to send */
          send(consumer, &m);                   /* send item to consumer */
     }
}

void consumer(void)
{
     int item, i;
     message m;

     for (i = 0; i < N; i++) send(producer, &m);  /* send N empties */
     while (TRUE) {
          receive(producer, &m);                /* get message containing item */
          item = extract_item(&m);              /* extract item from message */
          send(producer, &m);                   /* send back empty reply */
          consume_item(item);                   /* do something with the item */
     }
}
```